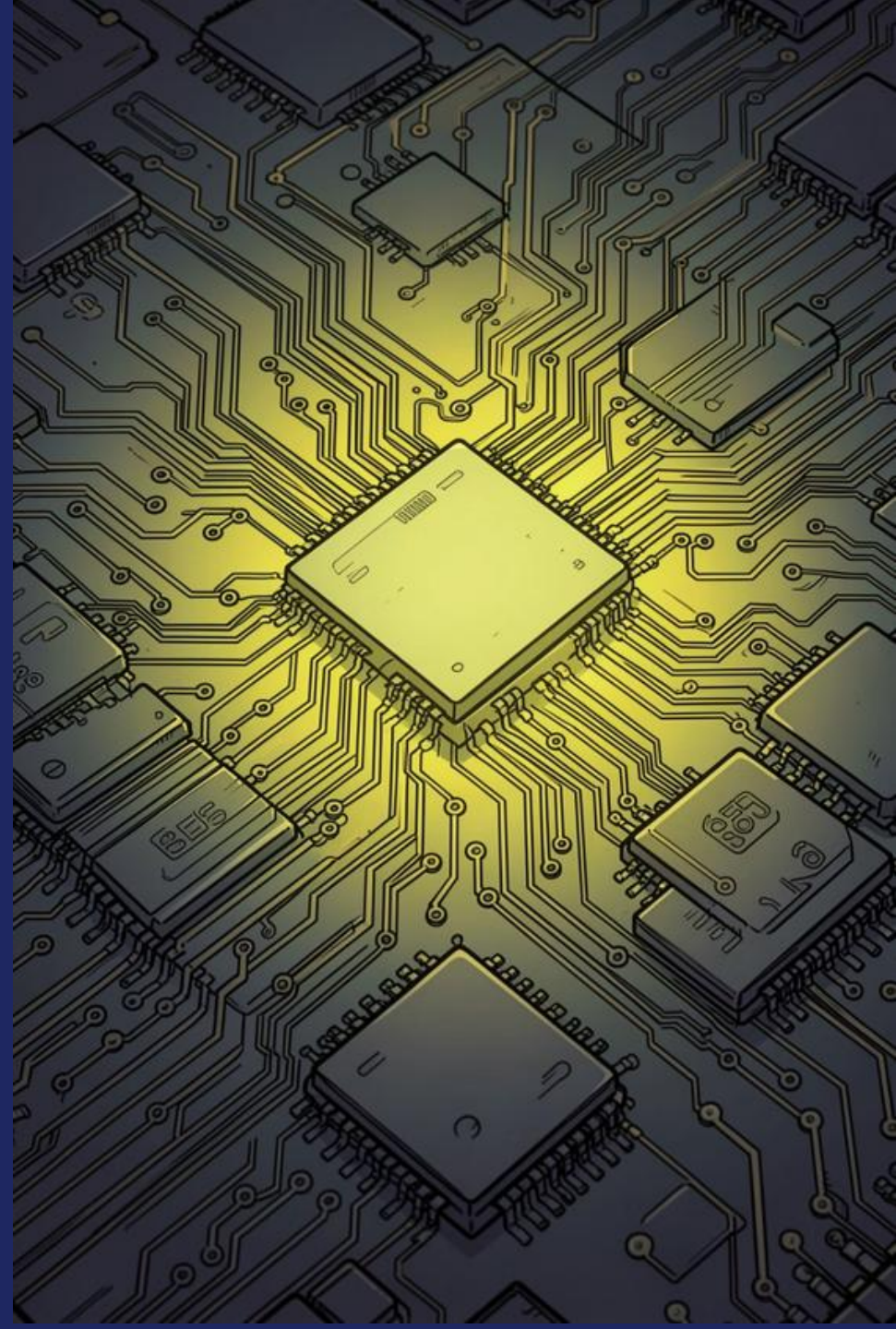
Multi-core: Global Scheduling

Md Mostafizur Rahman
Electronic Engineering,
Hochschule Hamm-Lippstadt (HSHL)

Seminar: Real-Time Systems · Prof. Stefan Henkler



Agenda

01 Motivation & Problem Statement

02 G-EDF vs. P-EDF: Background & Comparison

03 OS Overhead and WCET Inflation

04 EPOS vs. LITMUS^{RT}

05 WCET Inflation Formula

06 Local Experiment: Methodology & Results

07 Cross-Platform Comparison & Application Example

08 UPPAAL Formal Verification — Windows & Linux

09 Schedulability Analysis: Utilization vs. Deadline

10 Evaluation & Limitations

Motivation & Problem Statement

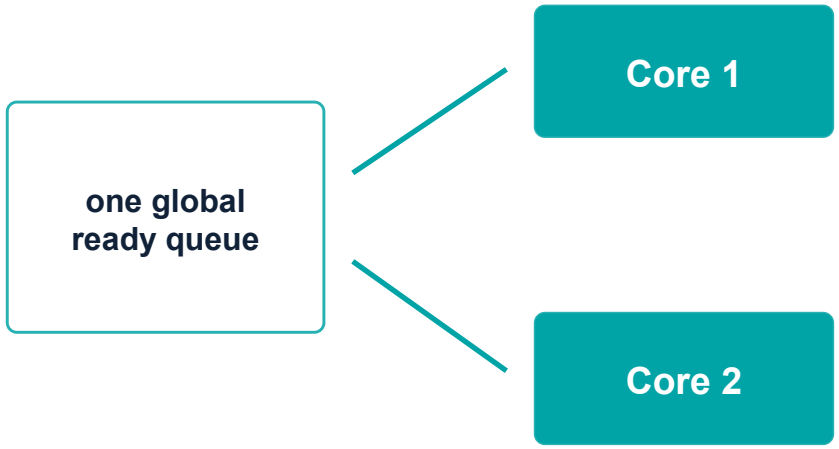
deadline

Why this matters

- Multicore chips are common in embedded and automotive systems.
- ADAS, braking control, sensor fusion, and vision tasks require deterministic timing.
- More cores increase computing power, but do not automatically guarantee deadlines.
- **OS overhead and interference inflate effective WCET.**

Global vs. Partitioned Scheduling

G-EDF



HOW IT WORKS

One global ready queue ordered by earliest deadline; the m earliest-deadline jobs run on the m cores.

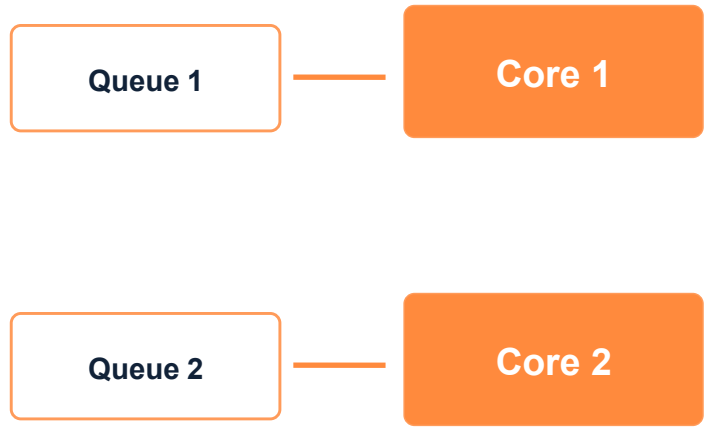
MAIN BENEFIT

Good load sharing — flexible use of all cores.

MAIN RISK

Migrations, IPI latency, cache-affinity loss, pessimistic schedulability tests.

P-EDF



HOW IT WORKS

Tasks are statically assigned to cores via bin-packing; each core runs EDF locally on its own task set.

MAIN BENEFIT

No migration after partitioning; often lower runtime interference.

MAIN RISK

NP-hard bin-packing can fail even when total utilization fits.

Src:Image Generated With Chatgpt 5.5

G-EDF Algorithm

1 Job release

Compute absolute_deadline = release_time + relative_deadline.

2 Enqueue globally

Insert the job into one global ready queue ordered by absolute deadline — O(n) insertion.

3 Select earliest m

Select the m jobs with earliest deadlines from the queue.

4 Dispatch

Dispatch the selected jobs to the available cores — O(1).

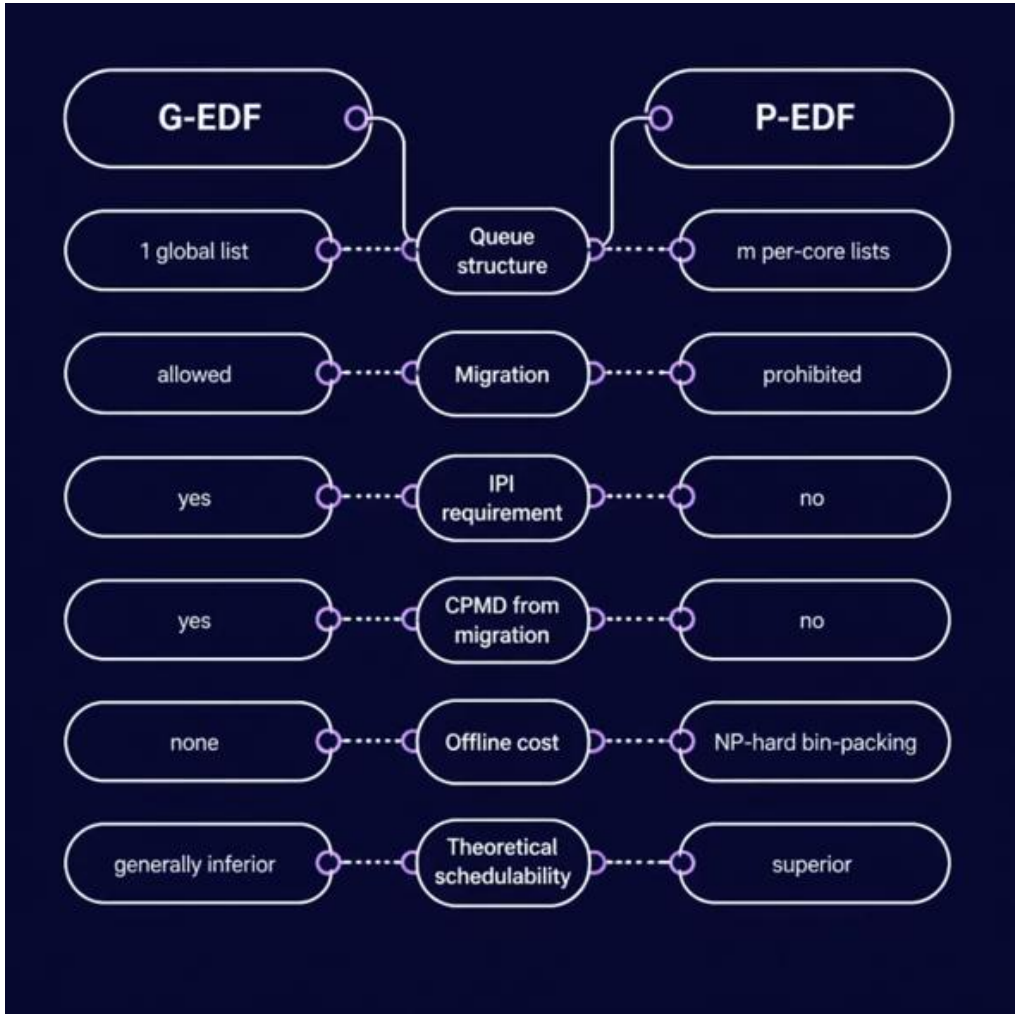
5 Preempt if needed

A newly released job with an earlier deadline preempts a running lower-priority job; an IPI triggers the remote reschedule.

6 Re-queue

The preempted job returns to the global queue and may resume on another core (migration).

G-EDF vs. P-EDF: Algorithm Comparison



Src:Image Generated With Chatgpt 5.5

OS Overhead and WCET Inflation

Overhead Source	Explanation	Relation to G-EDF
Context switch	Time to save/restore task context	Occurs when tasks are preempted or switched
Scheduling decision	Time to select the next job	Global queue cost grows with task count
Thread release	Time to make periodic jobs ready	Occurs when jobs are released
Tick counting	Timer interrupt handling overhead	Can disturb execution periodically
IPI latency	Delay for a cross-core reschedule request	Important in global scheduling
CPMD	Cache delay after preemption / migration	Important when jobs migrate between cores

$$\textit{inflated WCET} = \textit{original WCET} + \textit{overhead / interference}$$

EPOS vs. LITMUS^{RT}

Overhead Source	EPOS (WC)	LITMUS^{RT} (WC)	Factor
Context switch	0.30 µs	≤ 29.56 µs	76×
IPI latency	~0.30 µs	≤ 4.5 µs	15×
Scheduling G-EDF	O(n), stable < 75	Unpredictable	7×
Scheduling P-EDF	O(n/m), stable	Unpredictable	21×
Tick counting	0.25 µs (const)	Grows with n	84×

Table 3: worst-case overhead, Intel i7-2600, 100 runs across 5–125 tasks (Gracioli et al. [1])

- We see that with **low-overhead EPOS**, G-EDF outperforms P-EDF on LITMUSRT in light-utilization, short-period task sets reversing the theoretical ordering.
- OS design quality can determine the practical ordering of G-EDF and P-EDF, **independently of scheduling theory**.

WCET Inflation Formula

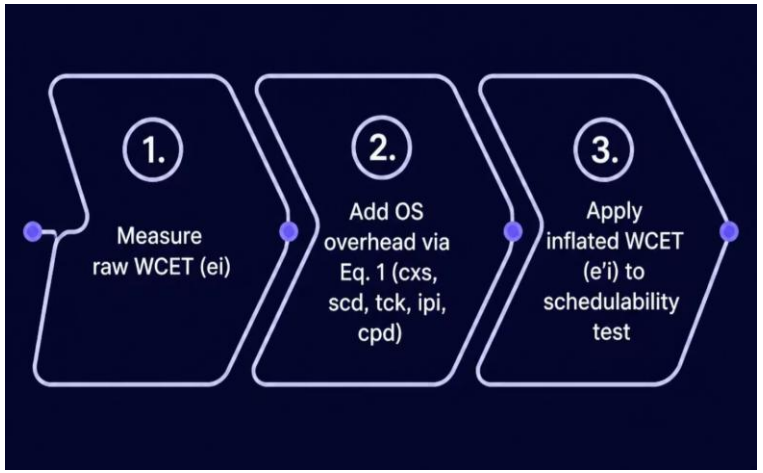
PREEMPTION-CENTRIC WCET INFLATION (Brandenburg [3], Eq. 1–2)

$$e_i' = e_i + \frac{2(scd + cxs) + cpd}{1 - u_{tck}^0} + 2c_{pre} + ipi$$

$$u_{tck}^0 = \frac{tck + rel}{Q}, \quad c_{pre} = \frac{tck + rel}{1 - u_{tck}^0}$$

Interpretation

- 2(scd+cxs): scheduler and context-switch cost around release/completion.
- cpd: cache-related delay after preemption or migration.
- ipi: treated as computation because G-EDF tests cannot handle self-suspension.
- the OS changes effective WCET, and WCET changes schedulability.



Src:Image Generated With Chatgpt 5.5

Experiment: Measuring WCET

Task under test: CPU-bound arithmetic loop — x += i^2 for 5,000,000 iterations

Platform: AMD Ryzen 7 7435HS (Lenovo LOQ 15ARP9) — 8 physical / 16 logical cores (SMT), 16 MB shared L3, 15.7 GB RAM. Same hardware, dual-booted: Windows 11 (build 10.0.26200.8655) and Linux Ubuntu.

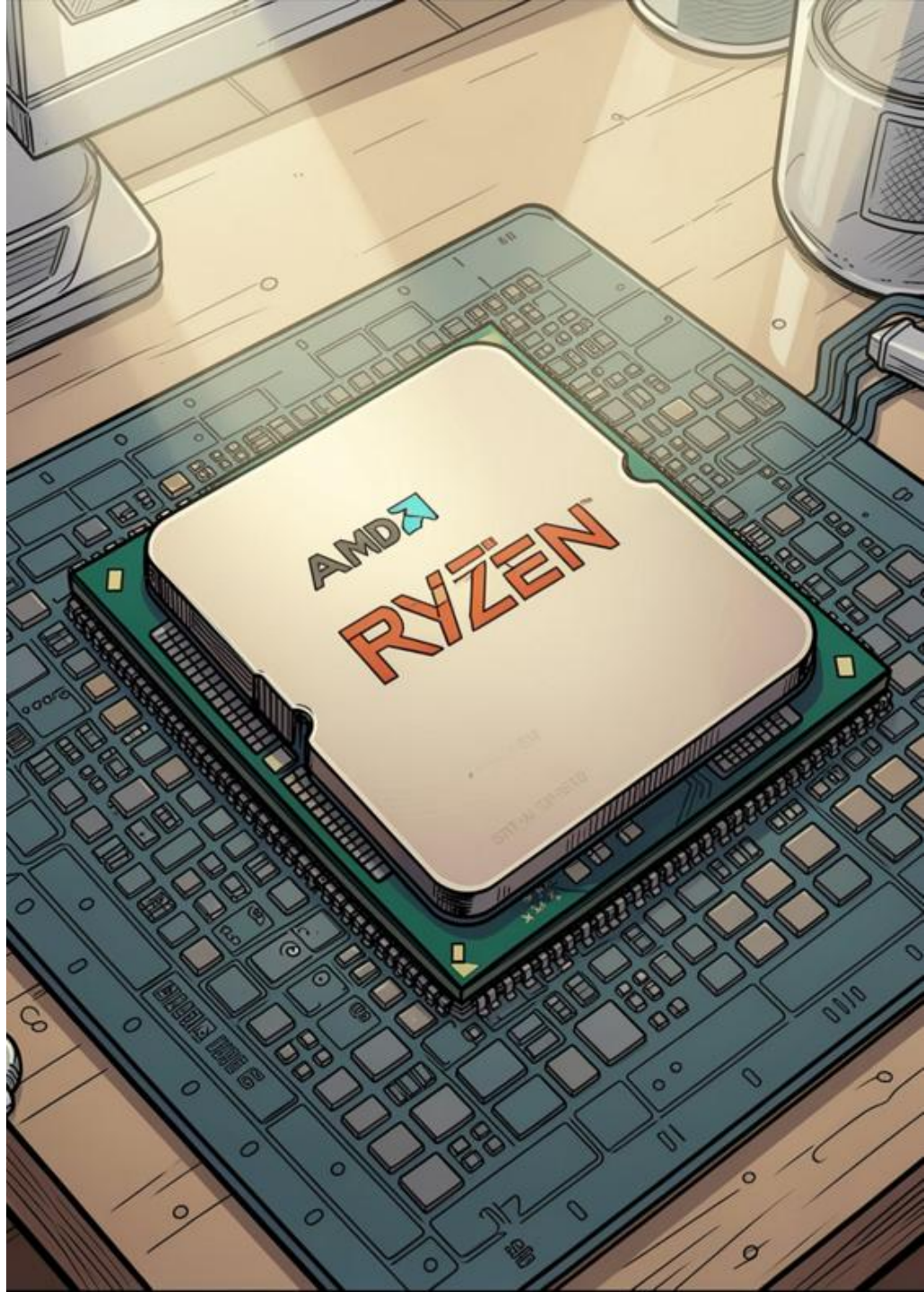
Phase 1 — No Load

- Run the task ten times with no background workers.
- Maximum observed runtime = sampled WCET (no load).

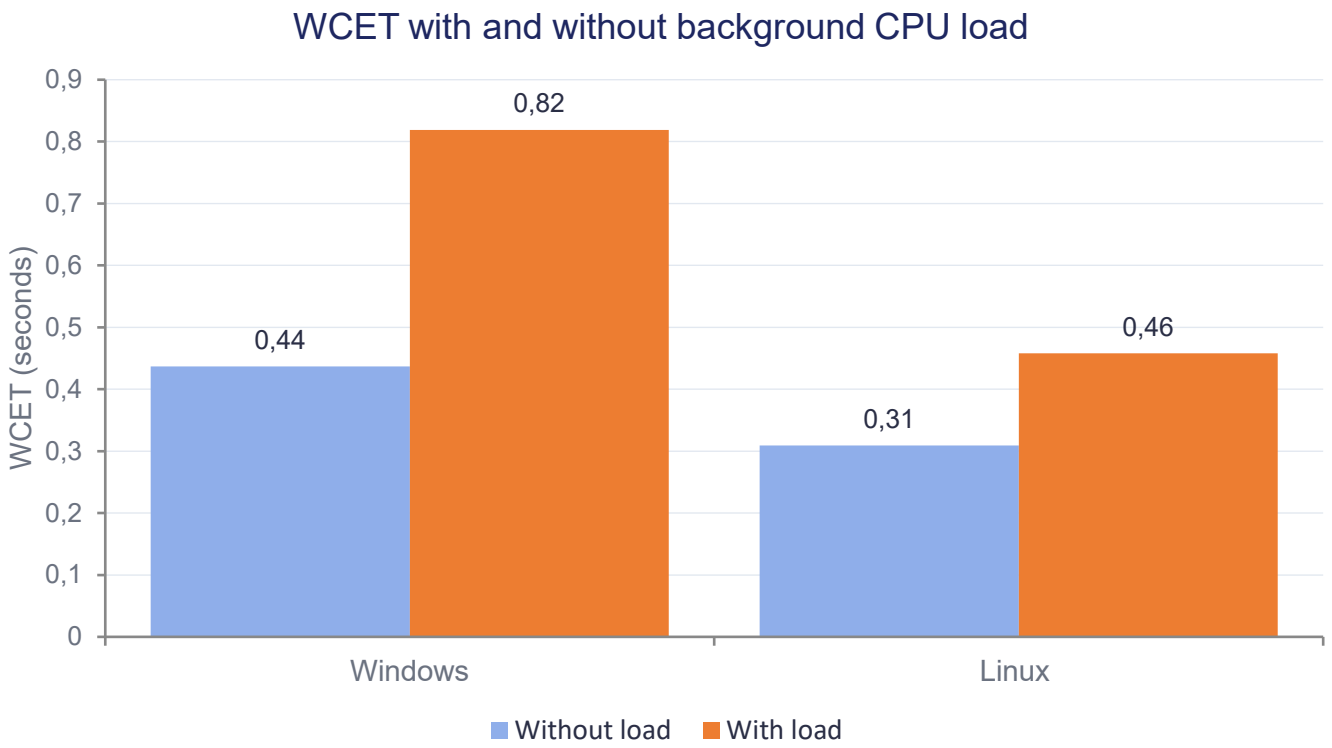
Phase 2 — Under Load

- Start cpu_count() − 1 = 15 worker processes across the 16 logical cores.
- Measure the same task ten more times; take the maximum runtime.

Linux perf extension: context switches, CPU migrations, cache references, cache misses, cycles, and instructions were recorded with perf stat for the full program execution.



Measurement Results: WCET Inflation



+87.3%

Windows
Increase under load (+0.382 s)

+48.5%

Linux
Increase under load (+0.150 s)

- OS-induced interference= CPU sharing + context switching + SMT/cache contention.

Linux Perf Counters (Loaded Run)

18,220

Context switches

Consistent with 15 worker processes competing across 16 logical cores.

13

CPU migrations

CFS keeps processes on their cores; very few migrations.

3.48 B

Cache references

Large amount of cache activity across the full run.

7.14 M

Cache misses

0.20% miss rate — hardware prefetcher keeps L3 hit rate high.

2.70

IPC

High IPC — execution is compute-bound, not memory-bound.

376 B

CPU cycles

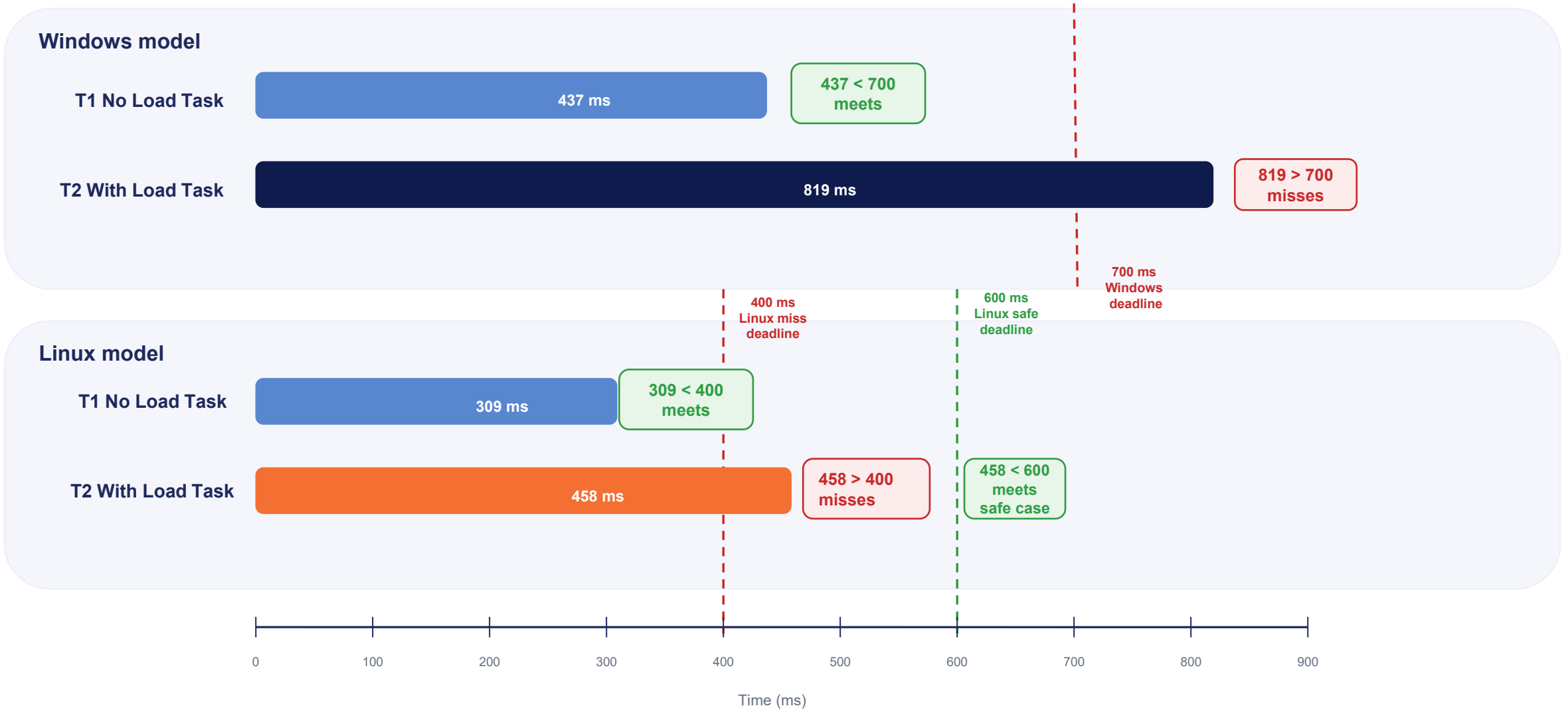
Total cycles counted for the full program run.

G-EDF Two-Core Application Example

Two tasks released simultaneously at t = 0 on a two-core system. Both cores are free, so G-EDF dispatches both immediately — no waiting, no preemption. The dispatch is always optimal; only the deadline choice determines the outcome.

Scenario	Task / WCET	Deadline	Outcome
Windows — safe (D_2=1000ms)	T1: 437 ms	D = 700 ms	**Meets deadline**
Windows — safe (D_2=1000ms)	T2: 819 ms	D = 1000 ms	**Meets deadline**
Windows — miss (D_2=700ms)	T1: 437 ms	D = 700 ms	**Meets deadline**
Windows — miss (D_2=700ms)	T2: 819 ms	D = 700 ms	**Misses deadline**
Linux — safe (D_2=600ms)	T1: 309 ms	D = 400 ms	**Meets deadline**
Linux — safe (D_2=600ms)	T2: 458 ms	D = 600 ms	**Meets deadline**
Linux — miss (D_2=400ms)	T1: 309 ms	D = 400 ms	**Meets deadline**
Linux — miss (D_2=400ms)	T2: 458 ms	D = 400 ms	**Misses deadline**

UPPAAL Model: Deadline Graph



Note: 600 ms is not measured runtime; it is only the chosen safe deadline. Since T2 = 458 ms is below 600 ms, the safe case meets the deadline.

Src:Image Generated With Chatgpt 5.5

Formal Verification Queries

UPPAAL Query	Verdict	Interpretation
`A[] !deadlock`	**true**	Self-loops guarantee time always advances in Done/Missed locations.
`E<> NoLoad.Done`	**true**	No-load task can reach Done at x = 437 ms.
`A[] !miss_no_load`	**true**	No-load task never misses: 437 ms < 700 ms in all paths.
`E<> WithLoad.Missed`	**true**	Loaded task reaches Missed at y = 700 ms.
`E<> miss_with_load`	**true**	A state with miss_with_load = true is reachable.
`A[] !miss_with_load [Windows]`	**false**	Counterexample: loaded task (819 ms) always misses the 700 ms deadline.
`A[] !safe_deadline_miss [Linux safe]`	**true**	Both tasks meet their deadlines (309<400, 458<600).
`A[] !miss_deadline_miss [Linux miss]`	**false**	Loaded task (458 ms) always misses the 400 ms deadline.

Schedulability Analysis: Utilization vs. Deadline Test

DEADLINE CHECK

Windows:

- ✓ Without Loaded, e=437ms < D=700ms → schedulable;
- ○ Loaded, e'=819ms > D=700ms → unschedulable

Linux:

- ✓ Without Loaded, e=309ms < D=400ms → schedulable;
- ○ Loaded, e'=458ms > D=400ms → unschedulable

UTILIZATION TEST (U ≤ 1)

Windows:

U=2×0.437=0.874<1 ✓;

U=2×0.819=1.638>1 ✗

Linux:

U=2×0.309=0.618<1 ✓;

U=2×0.458=0.916<1 ✓

⚠ CRITICAL DISTINCTION

For Linux under load, the utilization test PASSES (0.916 < 1.0) but the deadline check FAILS (458 ms > 400 ms).

CPU capacity can look enough, but a task can still miss its deadline.

Critical Evaluation & Limitations

1 GPOS, not RTOS

Windows and Linux are compared, not EPOS vs. LITMUS^{RT}; this is a qualitative transfer experiment

2 Statistical WCET

WCET is the max of 10 runs, not an analytically bounded value (IPET, cache, flow analysis). May underestimate the true WCET for rare events.

3 Full-program counters

Perf counters measure the whole program including background workers, not the isolated task.

4 UPPAAL templates

UPPAAL verifies chosen timing values, not a full multicore OS scheduler

References

[1] G. Gracioli, A. A. Fröhlich, R. Pellizzoni, and S. Fischmeister, “Implementation and evaluation of global and partitioned scheduling in a real-time OS,” Real-Time Systems, vol. 49, pp. 669–714, 2013.

[2] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” Journal of the ACM, vol. 20, no. 1, pp. 46–61, 1973.

[3] B. B. Brandenburg, “Scheduling and locking in multiprocessor real-time operating systems,” Ph.D. dissertation, University of North Carolina at Chapel Hill, 2011.

[4] A. Bastoni, B. B. Brandenburg, and J. H. Anderson, “Cache-related preemption and migration delays: empirical approximation and impact on schedulability,” in Proc. 6th Workshop on Operating Systems Platforms for Embedded Real-Time Applications, 2010.

[5] M. Bertogna and M. Cirinei, “Response-time analysis for globally scheduled symmetric multiprocessor platforms,” in Proc. IEEE Real-Time Systems Symposium (RTSS), 2007, pp. 149–160.

[6] Python experiment: multicore_wcet_test.py on AMD Ryzen 7 7435HS, Windows 11 build 10.0.26200.8655 and Linux Ubuntu. perf stat -e context-switches,cpu-migrations,cache-references,cache-misses,cycles,instructions.

[7] UPPAAL model: wcet_deadline_uppaal_final_nondeadlock.xml. Verified with UPPAAL 4.x. Models: Windows single-deadline model and Linux two-case model.